Nome e Cognome: _____ **Matricola:** _____

Istruzioni: Il tempo a disposizione per completare la prova è di **45 minuti**. Durante la prova, non è consentito consultare materiale didattico, né interagire con dispositivi elettronici o colleghi. Nel caso di domande a risposta multipla con più risposte corrette, ciascuna risposta corretta indicata vale una corrispondente frazione del punteggio complessivo, mentre le risposte errate comportano una pari penalizzazione. In nessun caso, il punteggio complessivo relativo a una domanda sarà negativo.

Es. **1.** **(2 punti)** Quali paragrafi seleziona il selettore `h2 + p` nel frammento seguente?

```
<div id="d">
  <h2 id="t">Titolo</h2>
  <p id="p1">Uno</p>
  <section id="s">
    <p id="p2">Due</p>
  </section>
  <p id="p3">Tre</p>
</div>
```

- [A] p1 e p3
- [B] Nessuno
- [**C**] Solo p1
- [D] p1, p2 e p3

Risposta: C. `+` seleziona il fratello **immediatamente** successivo: solo `p1` segue direttamente l'`h2`. *(Dispensa: cap. 3, «Combinatori»)*

Es. **2.** **(2 punti)** Che differenza c'è fra `display: none` e `visibility: hidden`?

- [A] `display: none` rimuove l'elemento dal DOM, `visibility: hidden` no.
- [**B**] Con `display: none` l'elemento non occupa spazio; con `visibility: hidden` sì.
- [C] Nessuna: sono due modi equivalenti di nascondere un elemento.
- [D] `visibility: hidden` nasconde anche il contenuto dei figli, `display: none` no.
- [E] Con `visibility: hidden` l'elemento non occupa spazio; con `display: none` sì.
- [F] `display: none` vale solo per gli elementi di blocco.

Risposta: B. Entrambi nascondono l'elemento, che resta nel DOM. Con `display: none` non viene generato alcun riquadro, quindi gli altri elementi ne occupano il posto; con `visibility: hidden` il riquadro c'è ma è invisibile, e lo spazio resta vuoto. *(Dispensa: cap. 4, «La proprietà display»)*

Es. **3.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const pet = { nome: "Fido" };
console.log(pet.razza);
console.log(pet.padrone?.nome);
console.log(pet.padrone.nome);
```

- [A] `undefined`, poi viene sollevato un `TypeError`.
- [**B**] `undefined`, `undefined`, poi viene sollevato un `TypeError`.
- [C] `undefined`, `undefined`, `undefined`.
- [D] `undefined`, `undefined`, poi viene sollevato un `ReferenceError`.

Risposta: B. Leggere una proprietà inesistente restituisce `undefined`. L'optional chaining `?.` si ferma se ciò che sta a sinistra è `null` o `undefined` e restituisce `undefined`. Senza, leggere `.nome` da `undefined` solleva un `TypeError` (non un `ReferenceError`: la variabile `pet` esiste). *(Dispensa: cap. 5, «Optional chaining»)*

SOLUZIONI

Es. **4.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
function Pet(nome) {
  this.nome = nome;
  this.corri = function () { return "corro"; };
}
Pet.prototype.saluta = function () { return "ciao " + this.nome; };
const a = new Pet("Fido");
const b = new Pet("Rex");
console.log(a.saluta === b.saluta, a.corri === b.corri);
console.log(b.saluta());
```

- A `false false`, ciao Rex.
- B `true false`, ciao Fido.
- C `false true`, ciao Rex.
- **D** `true false`, ciao Rex.
- E `true true`, ciao Rex.
- F `true false`, ciao `undefined`.

Risposta: D. I metodi definiti su `Pet.prototype` esistono una sola volta e sono condivisi da tutte le istanze tramite la catena dei prototipi. Una funzione assegnata a `this` nel costruttore viene invece ricreata per ogni oggetto. `this` dentro `saluta` è l'oggetto su cui la si chiama. *(Dispensa: cap. 6, «Due modi per aggiungere metodi»)*

Es. **5.** **(2 punti)** Si consideri il seguente frammento di documento HTML. Cosa viene stampato in console in seguito ad un click sul `<button>`?

```
<div id="d">
  <p id="p">
    <button id="b">Clic</button>
  </p>
</div>
<script>
  document.getElementById("p").addEventListener("click", () => console.log("p"));
  document.addEventListener("click", () => console.log("document"));
  document.getElementById("d").addEventListener("click", () => console.log("div"));
  document.getElementById("b").addEventListener("click", () => console.log("button"));
</script>
```

- A `document`, `button`, `p`, `div`.
- B `button`, `p`, `div`.
- C `p`, `document`, `div`, `button`.
- D `document`, `div`, `p`, `button`.
- **E** `button`, `p`, `div`, `document`.
- F `button`.

Risposta: E. Per il *bubbling*, un clic sul pulsante esegue prima i gestori del pulsante, poi quelli del suo genitore e così via, risalendo fino a `document`, qualunque sia l'ordine in cui i gestori sono stati registrati. `stopPropagation()` non interrompe il gestore in cui viene chiamato, ma impedisce all'evento di risalire oltre. *(Dispensa: cap. 7, «Bubbling» e «Fermare il bubbling»)*

Es. **6.** **(2 punti)** Un'applicazione gestisce `GET` e `POST` sul percorso `/todo`. Quale risposta è più corretta per una richiesta `DELETE /todo`?

- A `204 No Content`
- B `500 Internal Server Error`
- C `404 Not Found`
- D `200 OK` con la lista
- E `400 Bad Request`
- **F** `405 Method Not Allowed`

Risposta: F. Quando il percorso esiste ma il metodo non è previsto, la risposta corretta è 405, non 404: client e *proxy* sanno distinguere i due casi, e la distinzione torna utile soprattutto nelle API REST. *(Dispensa: cap. 12, «Instradare anche in base al metodo»)*

SOLUZIONI

Es. **7.** **(2 punti)** Un'applicazione memorizza nei cookie `auth=true` e `username=luigi` dopo il login, e controlla quei cookie a ogni richiesta. Quali delle seguenti affermazioni sono **vere**?

- [x] **A** I browser limitano la dimensione e il numero dei cookie per dominio.
- [x] **B** Con i cookie firmati dal server la manomissione si riconosce, ma i dati restano sul client.
- [ ] C I cookie non sono modificabili, perché li ha impostati il server.
- [ ] D Il browser invia i cookie solo alla pagina di login.
- [x] **E** Un utente può cambiare `username` e farsi riconoscere come un altro utente.
- [ ] F Il problema si risolve rendendo i valori più lunghi.

Risposte: A, B, E. I cookie sono interamente controllati dal client: dai DevTools si leggono e si modificano con un doppio clic. In più sono limitati (di solito 4096 byte ciascuno, da 20 a 50 per dominio). La soluzione sono le sessioni: i dati restano sul server, il cookie porta solo un identificatore. *(Dispensa: cap. 13, «I problemi dell'approccio a cookie»)*

Es. **8.** **(2 punti)** Un'immagine larga quanto il viewport ha `srcset="foto-400.jpg 400w, foto-800.jpg 800w, foto-1200.jpg 1200w"`. Quale file scaricherà presumibilmente uno smartphone con un viewport largo 375px e densità di pixel 2?

- [ ] A `foto-400.jpg`
- [ ] B Il file indicato in `src`, perché `srcset` vale solo sui desktop.
- [x] **C** `foto-800.jpg`
- [ ] D `foto-1200.jpg`

Risposta: C. Senza `sizes` l'immagine si considera larga `100vw`: servono $375 \cdot 2 = 750$ pixel, e il candidato più adatto è quello da 800. Il browser scarica solo il file scelto; `src` resta il valore di riserva per chi non supporta `srcset`. *(Dispensa: cap. 14, «srcset»)*

Es. **9.** **(5 punti)** Si confrontino cookie, `localStorage`, `sessionStorage` e IndexedDB. Quali dati non vi andrebbero mai memorizzati, e perché?

Traccia della risposta.

- Cookie: piccole stringhe impostate di solito dal server con `Set-Cookie` e rimandate dal browser in tutte le richieste verso lo stesso dominio; da JavaScript con `document.cookie`.
- `localStorage`: coppie chiave/valore (solo stringhe) non inviate al server, una per combinazione di dominio, protocollo e porta; sopravvivono al riavvio del browser.
- `sessionStorage`: stessa interfaccia, ma legato alla scheda (sopravvive all'aggiornamento, non alla chiusura). IndexedDB: un database nel browser, per le applicazioni offline.
- Sono sul computer dell'utente, ispezionabili e modificabili: niente segreti, e il server non deve fidarsi di questi dati (es. i prezzi di un carrello).

(Dispensa: cap. 8, «Memorizzare dati nel browser»)

Es. **10.** **(5 punti)** Si descriva il modello di esecuzione di Node.js, spiegando per quali applicazioni è adatto e per quali no.

Traccia della risposta.

- Un unico processo: il codice JavaScript gira in un solo thread, l'event loop, che preleva un evento alla volta dalla coda e ne esegue il gestore.
- Le operazioni di I/O sono asincrone: vengono affidate ai worker del runtime, e l'event loop passa subito all'evento successivo; a operazione conclusa la callback torna in coda.
- Adatto ai carichi dominati dall'I/O (molte richieste che aspettano database, file, rete): un solo thread ne serve moltissime.
- Poco adatto ai carichi dominati dalla CPU: un calcolo lungo e sincrono blocca l'event loop e quindi tutte le richieste.

(Dispensa: cap. 11, «Il modello di Node.js»)

SOLUZIONI

Es. **11.** **(5 punti)** Si confrontino `cookie-parser` ed `express-session`, e si spieghi a che cosa servono i messaggi flash e come funzionano.

Traccia della risposta.

- `cookie-parser` analizza l'header `Cookie` in `req.cookies`; con `res.cookie` si impostano cookie i cui valori restano sul client, che può modificarli.
- `express-session` tiene i dati sul server e mette nel cookie solo l'identificatore, firmato con il `secret` (manomissioni riconosciute); archivio in memoria solo per lo sviluppo.
- Messaggi flash: dopo un'azione si reindirizza, e il messaggio va conservato fino alla richiesta successiva, che è un nuovo ciclo.
- `connect-flash` li salva nella sessione (dopo `express-session`): `req.flash(k, msg)` li aggiunge, `req.flash(k)` li legge e li consuma.

(Dispensa: cap. 15, «Middleware utili» e «Messaggi flash»)

Griglia delle risposte

Es.	Risposta	Capitolo	Argomento
1	C	3	combinatori
2	B	4	display
3	B	5	oggetti
4	D	6	prototipi ed ereditarietà
5	E	7	propagazione degli eventi
6	F	12	routing
7	A, B, E	13	lo stato nei cookie
8	C	14	immagini responsive
9	aperta	8	memorizzare dati nel browser
10	aperta	11	il modello di Node.js
11	aperta	15	sessioni e cookie

Punteggio massimo: 16 (chiuse) + 15 (aperte) = **31 punti**. Nelle domande con più risposte corrette ogni risposta giusta indicata vale una frazione dei punti e ogni risposta sbagliata indicata ne toglie altrettanti; il punteggio di una domanda non scende mai sotto zero.

SOLUZIONI